

Assignment No. 04

- **TITLE** – To handle fragmented data efficiently, implement simple transpose, fast transpose, and addition of sparse matrices using the 3-tuple representation. Analyse and compare the time and space efficiency of each method to understand how sparse matrix operations can optimize performance in real-world scenarios like handling large scale data fragmentations on distributed systems.

➤ **THEORY** –

1. Sparse Matrices

A sparse matrix is a matrix in which most of the elements are zero. Storing all elements in a normal 2D array wastes both memory and computation time. Instead, only the non-zero elements are stored using a compact representation.

Applications of Sparse Matrices:

- Graph algorithms (adjacency matrices).
- Search engines and recommendation systems.
- Machine learning (large datasets with many missing values).
- Distributed systems where data is fragmented.

2. Triplet (3-Tuple) Representation

The triplet representation stores only the non-zero elements of a sparse matrix. Each element is represented as:

(row,column,value)(row, column, value)(row,column,value)

The first entry usually stores metadata:

- row = total number of rows
- col = total number of columns
- value = total number of non-zero elements

This reduces space complexity from $O(\text{rows} \times \text{cols})$ to $O(\text{nz})$, where nz = number of non-zero elements.

3. Sparse Matrix Operations

a) Simple Transpose

- Swaps row and column indices of each element.
- Implementation: Scan column by column, copy elements with swapped indices.
- Time Complexity: $O(\text{nz} \times \text{cols})$ (can be slow for large matrices).

b) Fast Transpose

- Improves efficiency by precomputing positions of elements using a count array.
- Steps:
 1. Count the number of elements in each column.
 2. Compute starting index for each column in the transposed matrix.
 3. Place elements directly in correct positions.
- Time Complexity: $O(\text{nz} + \text{cols}) \rightarrow$ faster than simple transpose.

c) Addition of Sparse Matrices

- Convert both matrices into triplet form.

- Compare positions (row, col) of non-zero elements:
 - If positions match → add values.
 - Else → insert element directly.
- Time Complexity: $O(nz1 + nz2)$ where $nz1$ and $nz2$ are non-zero counts.

4. Time and Space Efficiency

Operation	Normal Matrix (Dense)	Sparse Triplet Representation
Storage	$O(\text{rows} \times \text{cols})$	$O(nz)$
Simple Transpose	$O(\text{rows} \times \text{cols})$	$O(nz \times \text{cols})$
Fast Transpose	$O(\text{rows} \times \text{cols})$	$O(nz + \text{cols})$
Addition	$O(\text{rows} \times \text{cols})$	$O(nz1 + nz2)$

- Space Efficiency: Saves memory for large, mostly zero matrices.
- Time Efficiency: Operations are performed only on non-zero elements, reducing runtime.

5. Real-World Relevance

- Distributed Systems → Handle fragmented data efficiently by reducing network and memory overhead.
- Graph Algorithms → Store adjacency matrices compactly.
- Machine Learning → Work with high-dimensional but sparse datasets.
- Scientific Computing → Solve large systems with minimal storage.

➤ ALGORITHM –

A. Triplet Representation

1. Input matrix dimensions (rows, cols) and elements.
2. Count non-zero elements (nz).
3. Create a triplet array of size $(nz+1) \times 3$.
 - First row → (rows, cols, nz) (metadata).
 - Remaining rows → (row_index, col_index, value) for each non-zero element.
4. Print triplet form.

B. Simple Transpose

1. Initialize transposed triplet array with swapped dimensions.
2. For each column in original, scan all elements and if found → store (col, row, value).
3. Print result.

C. Fast Transpose

1. Create an array count[col] → store number of non-zero elements per column.
2. Compute start[col] → index where that column's elements should start in result.
3. Traverse original triplet and directly place (col, row, value) in correct index of transposed array.
4. Print result.

D. Addition of Sparse Matrices

1. Generate triplet representations of both matrices.
2. Compare elements of both triplets by (row, col).
 - If positions match → add values.
 - If not → copy smaller one directly.
3. Store results in new triplet array.
4. Print result.

➤ EXAMPLE –**Matrix A (4×5)**

```
0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0
```

Matrix B (4×5)

```
0 0 0 0 5
0 0 2 0 0
0 8 0 0 0
0 0 7 0 0
```

Triplet Representation of A

```
Row Col Val
4 5 6 <- (rows, cols, non-zero count)
0 2 3
0 4 4
1 2 5
1 3 7
3 1 2
3 2 6
```

Triplet Representation of B

```
Row Col Val
4 5 5
0 4 5
1 2 2
2 1 8
3 2 7
```

Simple Transpose of A

```
Row Col Val
5 4 6
2 0 3
4 0 4
2 1 5
3 1 7
1 3 2
2 3 6
```

Fast Transpose of A

```
Row Col Val
5 4 6
1 3 2
2 0 3
2 1 5
2 3 6
3 1 7
4 0 4
```

Addition of A + B

Row	Col	Val
4	5	8
0	2	3
0	4	9
1	2	7
1	3	7
2	1	8
3	1	2
3	2	13

➤ **CODE –****1. sparse_matrix.cpp**

```
#include <iostream>
using namespace std;

struct Triple{
    int row;
    int col;
    int value;
};

int main(){
    int rows, cols, nz=0;
    cout<<"Enter number of rows and columns respectively: "<<endl;
    cin>>rows>>cols;
    int matrix[rows][cols];
    cout<<"Enter elemens of matrix: "<<endl;
    for(int i = 0; i < rows; i++){
        for(int j = 0; j < cols; j++){
            cin>>matrix[i][j];
            if(matrix[i][j]!=0){
                nz++;
            }
        }
    }

    Triple sparse[nz+1];
    sparse[0].row = rows;
    sparse[0].col = cols;
    sparse[0].value = nz;

    int k = 1;
    for(int i = 0; i < rows; i++){
        for(int j = 0; j < cols; j++){
            if(matrix[i][j]!=0){
                sparse[k].row = i;
                sparse[k].col = j;
                sparse[k].value = matrix[i][j];
                k++;
            }
        }
    }

    cout<<"The sparse matrix is: "<<endl;
    cout << "Row\tCol\tValue" <<endl;
    for(int i = 1; i <= nz; i++){
        cout<<sparse[i].row<<"\t " <<sparse[i].col<<" \t" <<sparse[i].value<<endl;
    }
}
```

```

Triple simple[nz+1];
simple[0].row = sparse[0].col;
simple[0].col = sparse[0].row;
simple[0].value = sparse[0].value;

int t = 1;
for(int col = 0; col < cols; col++) {
    for(int i = 1; i <= nz; i++) {
        if(sparse[i].col == col) {
            simple[t].row = sparse[i].col;
            simple[t].col = sparse[i].row;
            simple[t].value = sparse[i].value;
            t++;
        }
    }
}

cout << "The simple transpose sparse matrix is:" << endl;
cout << "Row\tCol\tValue" << endl;
for(int i = 1; i <= nz; i++) {
    cout << simple[i].row << "\t" << simple[i].col << "\t" << simple[i].value <<
endl;
}

Triple fast[nz + 1];
fast[0] = {cols, rows, nz};

int count[cols] = {0}, index[cols];

for (int i = 1; i <= nz; i++){
    count[sparse[i].col]++;
}

index[0] = 1;
for (int i = 1; i < cols; i++){
    index[i] = index[i - 1] + count[i - 1];
}

for (int i = 1; i <= nz; i++) {
    int col = sparse[i].col;
    int pos = index[col]++;
    fast[pos] = {sparse[i].col, sparse[i].row, sparse[i].value};
}

cout << "The fast transpose sparse matrix is:" << endl;
cout << "\nRow\tCol\tValue\n";
for (int i = 1; i <= nz; i++){
    cout << fast[i].row << "\t" << fast[i].col << "\t" << fast[i].value << endl;
}
return 0;
}

```

2. AddSparse.cpp

```

#include <iostream>
using namespace std;
struct Triple{
    int row;
    int col;

```

```

    int value;
};
int main(){
    int r1, r2, c1, c2;
    cout<<"Enter number of rows and columns of first matrix respectively: "<<endl;
    cin>>r1>>c1;
    cout<<"Enter number of rows and columns of second matrix respectively: "<<endl;
    cin>>r2>>c2;
    if(c1!=c2||r1!=r2){
        cout<<"Matrix multiplication not possible"<<endl;
        return 0;
    }
    int matrix1[r1][c1], matrix2[r2][c2];
    cout<<"Enter elements of first matrix: "<<endl;
    for(int i = 0; i < r1; i++){
        for(int j = 0; j < c1; j++){
            cin>>matrix1[i][j];
        }
    }
    cout<<"Enter elements of second matrix: "<<endl;
    for(int i = 0; i < r2; i++){
        for(int j = 0; j < c2; j++){
            cin>>matrix2[i][j];
        }
    }
    int nz1=0, nz2=0;
    for(int i = 0; i < r1; i++){
        for(int j = 0; j < c1; j++){
            if(matrix1[i][j]!=0){
                nz1++;
            }
            if(matrix2[i][j]!=0){
                nz2++;
            }
        }
    }
    Triple sparse1[nz1+1], sparse2[nz2+1];
    sparse1[0].row = r1;
    sparse1[0].col = c1;
    sparse1[0].value = nz1;
    sparse2[0].row = r2;
    sparse2[0].col = c2;
    sparse2[0].value = nz2;

    int k = 1;
    for(int i = 0; i < r1; i++){
        for(int j = 0; j < c1; j++){
            if(matrix1[i][j]!=0){
                sparse1[k].row = i;
                sparse1[k].col = j;
                sparse1[k].value = matrix1[i][j];
                k++;
            }
        }
    }

```

```

    }
}
k = 1;
for(int i = 0; i < r2; i++){
    for(int j = 0; j < c2; j++){
        if(matrix2[i][j] != 0){
            sparse2[k].row = i;
            sparse2[k].col = j;
            sparse2[k].value = matrix2[i][j];
            k++;
        }
    }
}

cout<<"The sparse matrix 1 is: "<<endl;
cout << "Row\tCol\tValue" <<endl;
for(int i = 1; i <= nz1; i++){
    cout<<sparse1[i].row<<"\t "<<sparse1[i].col<<" \t"<<sparse1[i].value<<endl;
}
cout<<"The sparse matrix 2 is: "<<endl;
cout << "Row\tCol\tValue" <<endl;
for(int i = 1; i <= nz2; i++){
    cout<<sparse2[i].row<<"\t "<<sparse2[i].col<<" \t"<<sparse2[i].value<<endl;
}

Triple result[nz1+nz2+1];
result[0].row = r1;
result[0].col = c1;
int p=1;
for(int i=1; i<=nz1; i++){
    result[p]=sparse1[i];
    p++;
}
for(int i=1; i<=nz2; i++){
    bool found=false;
    for(int j=1; j<p; j++){
        if(sparse2[i].row==result[j].row && sparse2[i].col==result[j].col){
            result[j].value+=sparse2[i].value;
            found=true;
            break;
        }
    }
    if(!found){
        result[p]=sparse2[i];
        p++;
    }
}
result[0].value=p-1;
cout<<"The resultant sparse matrix after addition is: "<<endl;
cout << "Row\tCol\tValue" <<endl;
for(int i = 1; i < p; i++){
    cout<<result[i].row<<"\t "<<result[i].col<<" \t"<<result[i].value<<endl;
}

```

```

    }
    return 0;
}

```

➤ OUTPUT –

```

Enter number of rows and columns respectively:
3 3
Enter elements of matrix:
0 0 2
4 5 0
9 0 0
The sparse matrix is:
Row    Col    Value
0      2      2
1      0      4
1      1      5
2      0      9
The simple transpose sparse matrix is:
Row    Col    Value
0      1      4
0      2      9
1      1      5
2      0      2
The fast transpose sparse matrix is:
Row    Col    Value
0      1      4
0      2      9
1      1      5
2      0      2

```

```

Enter number of rows and columns of first matrix respectively:
3 3
Enter number of rows and columns of second matrix respectively:
3 3
Enter elements of first matrix:
0 0 2
3 5 0
0 7 0
Enter elements of second matrix:
1 0 0
4 5 0
2 0 0
The sparse matrix 1 is:
Row    Col    Value
0      2      2
1      0      3
1      1      5
2      1      7
The sparse matrix 2 is:
Row    Col    Value
0      0      1
1      0      4
1      1      5
2      0      2
The resultant sparse matrix after addition is:
Row    Col    Value
0      2      2
1      0      7
1      1      10
2      1      7
0      0      1
2      0      2

```

➤ CONCLUSION –

In this practical, we have observed that Sparse matrices allow efficient storage and operations by focusing only on non-zero elements. Using the triplet representation, operations like fast transpose and addition reduce both memory usage and computation time, making them highly suitable for large-scale and real-world applications.